

Prova Finale: Progetto di Reti Logiche

Michele Conficconi 10844260,
Christian Corvace 10829358

August 11, 2025



POLITECNICO
MILANO 1863

1 Introduzione

L'obiettivo di questo progetto è implementare un filtro differenziale descrivendo l'architettura e i testbench in **VHDL**, utilizzando lo strumento di sintesi **XILINX VIVADO WEBPACK 2018.3** [1], e come FPGA designata una **Artix-7 FPGA xc7a200tbg484-1**.

Il filtro intende elaborare una sequenza di numeri prelevata dalla memoria attraverso moltiplicazioni, somme e normalizzazioni, riscrivendo il risultato nella memoria stessa. Lo **scopo** è un utilizzo corretto, consapevole ed efficiente dello strumento di sintesi.

I **risultati raggiunti** sono stati un corretto funzionamento del componente (secondo il comportamento descritto nella specifica data) sia in pre-sintesi che in post-sintesi, con l'utilizzo di soli registri flip-flop e un worst slack time di 12.242 ns.

La **relazione** inizia descrivendo il filtro e come abbiamo rielaborato la specifica assegnata (Sezione 2). Dopodiché, verranno illustrate le scelte progettuali e di implementazione dell'architettura del modulo e del suo funzionamento (Sezione 3). Successivamente verranno analizzati i risultati funzionali, in cui il componente viene testato sia in condizioni di normale funzionamento sia in specifici corner-case (Sezione 4). Seguiranno le analisi del componente sintetizzato, con riferimento alle risorse utilizzate e dello slack time (Sezione 5). Infine sono presenti le conclusioni (Sezione 6) e l'appendice (Sezione A).

2 Contesto e Descrizione del Filtro

Secondo la specifica, l'**obiettivo** del filtro è elaborare una sequenza di numeri di lunghezza prefissata **K**, attraverso dei coefficienti **C_j**. Il filtro ha due possibili configurazioni, ordine **3** e ordine **5**; per entrambe le configurazioni e per una qualsiasi i-esima parola viene eseguita la seguente funzione:

$$f'(i) = \frac{1}{n} \sum_{j=-l}^l (C_j \times f[i+j]) \quad (1)$$

I numeri adiacenti all'i-esima parola (da **-l** a **+l**) vengono moltiplicati per i coefficienti, sommati e normalizzati. Il **coefficiente di normalizzazione** $\frac{1}{n}$ e il valore di **l** variano a seconda dell'ordine: n vale $\frac{1}{12}$ per l'ordine 3 e $\frac{1}{60}$ per l'ordine 5, invece **l** vale 2 per l'ordine 3 e 3 per l'ordine 5.

Il componente deve interfacciarsi con una **memoria RAM**, che contiene dei dati secondo la disposizione illustrata in figura 1. Più nel dettaglio, la RAM contiene una **sequenza di inizializzazione** di 17 parole seguita da K

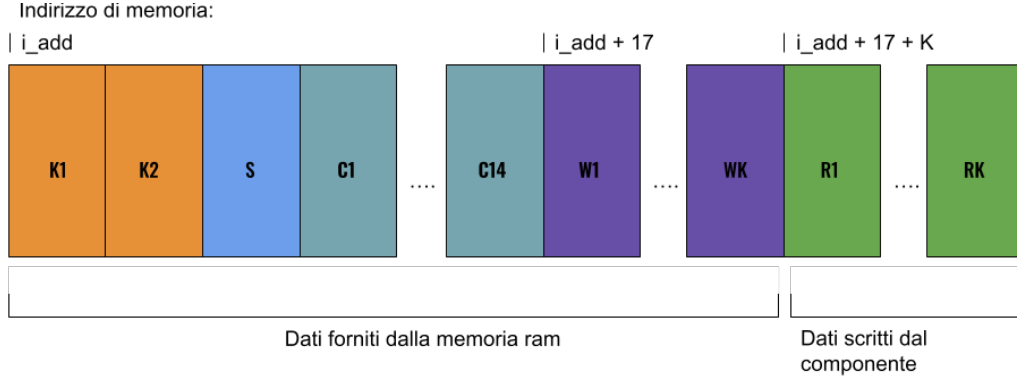


Figure 1: Disposizione della sequenza di dati in memoria

parole **W** da elaborare e K risultati finali **R**. Ogni parola corrisponde a un byte e i valori sono rappresentati in complemento a 2.

La sequenza di inizializzazione comprende i seguenti byte:

1. K1 e K2: indicano la lunghezza della sequenza di parole da elaborare ($K=K1.K2$)
2. S: indica l'ordine del filtro da selezionare attraverso il bit meno significativo, '0' indica l'ordine 3 e '1' indica l'ordine 5
3. Da C_1 a C_{14} : indicano i coefficienti dei due filtri, i primi 7 per l'ordine 3 e gli ultimi 7 per l'ordine 5, assumendo che C_1 e C_7 siano sempre 0
4. K: sequenza di parole da elaborare con valore compreso tra -128 e +127

Alla fine delle K parole verranno scritti, progressivamente e uno alla volta, i valori elaborati (denominati R_i in figura 1).

Se il risultato post normalizzazione eccede l'intervallo $[-128, +127]$ viene **saturato** ai limiti di tale intervallo. i_add rappresenta l'indirizzo base dell'inizio della memoria RAM, viene fornito come input ed è stabile durante tutta l'elaborazione della sequenza. Siccome vengono trattati numeri interi binari, la divisione per il coefficiente di normalizzazione può essere **approssimata** nel seguente modo:

1. $\frac{1}{12}$ è approssimato come $\frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}$
2. $\frac{1}{60}$ è approssimato come $\frac{1}{64} + \frac{1}{1024}$

Essendo i denominatori tutti potenze di due, questa operazione può essere effettuata velocemente con uno **shift logico** verso destra. Tuttavia, se il numero da normalizzare è **negativo**, lo shift logico produce un troncamento

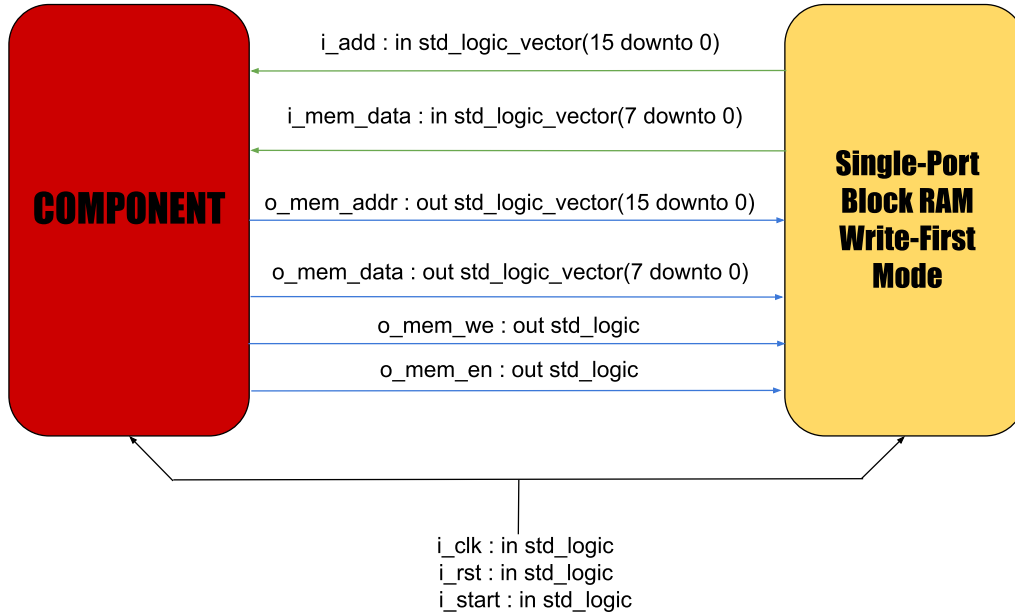


Figure 2: Interazione tra FSM e memoria RAM

verso -1 , che equivale a una diminuzione di precisione e quindi a un **errore**. Shift logici successivi accumulano errore sempre più grande, e per ridurlo, dopo ogni operazione di shift logico di un numero negativo, viene sommato $+1$.

È presente anche un **caso particolare** molto importante: siccome per ogni i -esima parola il filtro cerca di leggere i byte adiacenti da $-l$ a $+l$, quando l'indice delle parole da elaborare è subito dopo i 17 byte di inizializzazione o alla fine delle K parole, i valori prima della parola (se all'inizio), o dopo la parola (se alla fine), sono al di fuori della memoria dedicata per le K parole e vanno considerati pari a zero. Questo succede quando il filtro cerca di prelevare parole con indirizzo ' $i_add + 17 - l$ ' (con $l \in [-3, 0)$ per l'ordine 5, oppure $l \in [-2, 0)$ per l'ordine 3) ad inizio sequenza, oppure con indirizzo ' $i_add + 17 + K + l$ ' (con $l \in (0, +3]$ per l'ordine 5, oppure $l \in (0, +2]$ per l'ordine 3) a fine sequenza.

Lo schema a blocchi dell'interazione tra la nostra architettura e la memoria è descritto nella figura 2, dove vengono illustrati i segnali scambiati tra i due componenti.

La memoria funziona sia in **lettura** che in **scrittura** tramite i seguenti segnali: ***o_mem_en***, che abilita la comunicazione, e ***o_mem_we***, che indica la modalità: '0' per la lettura, '1' per la scrittura. Il segnale ***i_add*** indica, come accennato prima, l'indirizzo base della sequenza dei dati di inizializ-

zazione del filtro in memoria e rimane stabile durante tutta l'elaborazione.

Se si vuole eseguire una lettura, si mette il segnale *o_mem_en* alto, il segnale *o_mem_we* basso e si specifica tramite ***o_mem_addr*** l'indirizzo della parola che si vuole leggere. Il dato verrà in seguito presentato al componente tramite ***i_mem_data***, leggibile dal ciclo di clock successivo.

Se si vuole eseguire una scrittura si alzano sia *o_mem_en* sia *o_mem_we*, si specifica l'indirizzo a cui si desidera scrivere tramite *o_mem_addr*, e il dato da scrivere tramite ***o_mem_data***.

Tutto il sistema funziona in modo **sincrono** sul fronte di salita del segnale di clock ***i_clk*** ad eccezione del segnale di reset ***i_rst*** che è **asincrono**.

Il sistema inizia l'elaborazione solo quando il segnale ***i_start*** viene portato a uno; tuttavia, secondo specifica, prima del primo start nella prima elaborazione verrà sempre dato un impulso del segnale di reset. Non è necessario invece per elaborazioni successive ricevere un impulso di reset. L'uscita primaria ***o_done*** deve essere '0' all'inizio dell'elaborazione e viene portata alta (e mantenuta tale finché *i_start* non torna a '0') al termine della computazione e della scrittura del risultato in memoria. Il segnale *i_start* rimarrà alto durante tutta l'elaborazione, e verrà riportato a zero una volta che viene notificata la fine della computazione (*o_done*='1').

Ecco un **esempio** su come dovrebbe funzionare il filtro.

I byte di configurazione sono:

1. K1=0, K2=12 (K=K1.K2=12)
2. S=14 (Ordine 3)
3. C_1-C_{14} : [0, 8, -8, 56, 5, -4, 0, 2, -47, 115, 84, -10, -98, -28]
4. La sequenza di K parole: [13, 8, 2, 3, -7, -23, 9, -1, -7, -7, 5, 2]

Provando a elaborare per esempio -23, essendo il filtro di ordine 3, i valori che dobbiamo considerare sono quelli che partono da due posizioni prima di -23 fino ad arrivare alle due posizioni successive a -23 (l=2). Tutti i numeri devono essere moltiplicati per i rispettivi coefficienti, da C_2 a C_6 . Viene svolta quindi la seguente operazione:

$$(3 \times 8) + (-7 \times -8) + (-23 \times 56) + (9 \times 5) + (-1 \times -4) = -1159$$

Successivamente il risultato viene normalizzato dividendo per 12 (approssimando con shift logico), ottenendo -95. Ripetendo questo procedimento per tutta la sequenza otteniamo 60, 27, 15, 22, -44, -95, 52, -24, -28, -26, 23, 1.

3 Scelte progettuali di Implementazione

La nostra architettura si compone di un unico modulo realizzato tramite una **Finished State Machine (FSM)**, come illustrato in figura 2 tramite i segnali precedentemente descritti, come da specifica.

Per illustrare in modo compatto il funzionamento del filtro, riportiamo un possibile pseudo-codice dell'algoritmo implementato.

Algorithm 1)]Pseudo codice dell'algoritmo(k1, k2, s, c1, ..., c14, V[K])

```
1 K = K1.K2; Ord = s;
2 if (Ord == 0)
3     for (int i=1; i<8; i++) {
4         Ci = ci;
5     }
6 else {
7     for (int i=1; i<8; i++) {
8         Ci = c(i+7);
9     }
10 }
11 int p1, ..., p7, SUM, SUM_SHIFT;
12 for (int delta_add = 17; delta_add < (K+17); delta_add++){
13     for (int j=0; j<7; j++){
14         pi = ci * V[delta_add + j - 3];
15     }
16     SUM = p1 + p2 + p3 + p4 + p5 + p6 + p7;
17     if (Ord == 0) {SUM_SHIFT = SUM * 1/12; }
18     else {SUM_SHIFT = SUM * 1/60; }
19     V[K + delta_add] = SUM_SHIFT;
20 }
```

Lo pseudocodice 3 ha una **complessità temporale** $\mathcal{O}(n)$, dove n è la lunghezza del vettore di input. Tale valore dipende da K , lunghezza del vettore di parole da processare, più 17 byte di configurazione iniziale, ($n = K + 17$), come anche riportato in Sezione 3.

Abbiamo optato per l'implementazione di un'unica FSM per un risultato compatto e chiaro, in modo da avere solo due processi:

- **delta_function: process(i_clk, i_rst)**: processo sequenziale che regola il meccanismo del cambio di stato
- **lambda_function: process(curr_state)**: processo combinatorio che descrive il comportamento di ogni stato

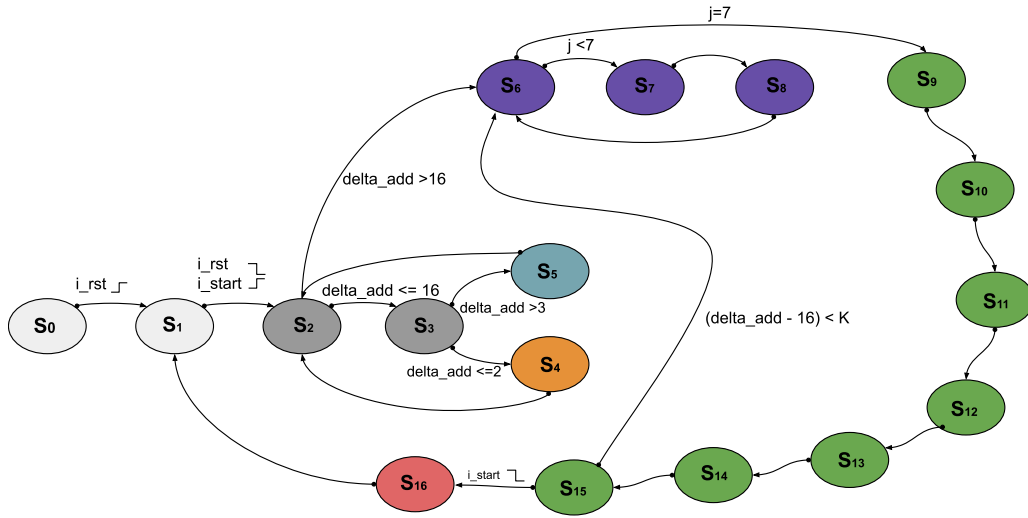


Figure 3: Schema degli stati della FSM, con condizioni di salto diverse dal segnale di clock

Per fare in modo che i segnali vengano salvati correttamente in flip-flop, e quindi aggiornati in modo sincrono, ognuno di essi presenta **due versioni**, una **sequenziale** (denominata tramite "_seq" alla fine del suo nome nel codice vhd) e una **combinatoria**. La versione sequenziale viene aggiornata unicamente all'interno della *delta_function* assegnandovi il corrispettivo valore combinatorio. Quella combinatoria invece viene modificata all'interno della *lambda_function* in due diversi modi: attraverso un calcolo/assegnamento, oppure tramite il riassegnamento del corrispettivo valore sequenziale, quando si vuole semplicemente mantenere il valore del segnale.

Lo schema della FSM è rappresentato dalla figura 3 e viene analizzato nel dettaglio nelle sezioni successive. In particolare, la sezione 3.1 spiega il funzionamento degli stati, la sezione 3.2 spiega le transizioni complesse che variano a seconda del valore dei segnali e infine la sezione 3.3 dettaglia la corrispondenza tra segnale e funzionalità.

3.1 Comportamento degli stati

Raggruppiamo e descriviamo gli stati in base alla loro **funzionalità**:

- **S_0, S_1** : Rappresentano gli stati che **precedono** l'inizio della computazione; S_0 precede il reset e S_1 precede il primo start. Inoltre, essendo S_1 lo stato che viene raggiunto dopo un reset, esso **resetta** tutti i segnali combinatori e sequenziali.

- S_2, S_3, S_4, S_5 : Questi stati prelevano dalla memoria ram tutti i byte di **configurazione** iniziale. S_2 effettua una richiesta di lettura alla memoria, S_3 è uno stato di attesa per la stabilizzazione dei segnali da memoria, S_4 salva la lunghezza della sequenza (K) e l'ordine del filtro (S), S_5 salva invece i coefficienti $C_1 - C_7$ o $C_8 - C_{14}$ in base all'ordine del filtro. Visto che, per il filtro di ordine 3, C_1 e C_7 non vengono mai realmente usati, verranno assunti sempre pari a 0.
- S_6, S_7, S_8 : Questi stati prelevano le parole da $-l$ a $+l$ e salvano il **prodotto parziale** per il corrispettivo coefficiente C_i . S_6 effettua una richiesta di lettura, S_7 è uno stato di attesa mentre S_8 riceve il dato dalla memoria e salva il prodotto, gestendo il caso limite in cui le parole da analizzare siano oltre l'inizio o la fine della sequenza.
- S_9, S_{10} : Calcolano la **somma** dei prodotti parziali precedentemente salvati nello stato S_8 .
- S_{11}, S_{12}, S_{13} : Calcolano lo **shift logico** della somma con eventuale correzione dell'approssimazione per i numeri negativi.
- S_{14}, S_{15} : Effettuano una **richiesta di scrittura** in memoria; se la computazione è finita, S_{15} alza il segnale di done, altrimenti torna allo stato S_6 per continuare l'elaborazione all'indirizzo successivo.
- S_{16} : Abbassa il segnale di done.

Nel codice della FSM, l'indice l (dall'equazione 1) che scorre le parole adiacenti alla i -esima parola considerata è stato nominato j e va da 0 a 7 per semplicità. Inoltre, per poter riutilizzare gli stessi segnali, i coefficienti C_i sono numerati solo da 1 a 7, poiché conoscendo il valore di S è possibile salvare direttamente i soli coefficienti da usare saltando all'indirizzo desiderato (solo da C_1 a C_7 per ordine 3 o solo da C_8 a C_{14} per l'ordine 5).

3.2 Transizione degli stati

Nella figura 3 vengono riportate le condizioni di cambio di stato diverse dal semplice fronte di salita del segnale di clock:

- Da S_0 a S_1 è necessario un fronte di salita del segnale di reset.
- Da S_1 a S_2 è necessario che il reset torni a zero e che start venga alzato.

- S_2 va in S_3 se deve ancora finire di salvare i byte di configurazione iniziali, altrimenti va in S_6 per procedere con l'elaborazione.
- S_3 va in S_4 se deve finire di salvare il segnale K o S, oppure va in S_5 se deve salvare i coefficienti C_i .
- S_6 va in S_7 se deve ancora prelevare ed elaborare parole da $-l$ a $+l$ (per fare il prodotto parziale tra parola e corrispettivo coefficiente), oppure in S_9 se le ha prelevate tutte e deve computare la somma parziale.
- S_{15} torna in S_6 se *non* sono state ancora elaborate tutte le parole della sequenza, altrimenti attende che il segnale di start venga abbassato per andare in S_{16} .

3.3 Dettaglio Implementativo sui Segnali della FSM

Come precedentemente detto ogni segnale ha due versioni, una **combinatoria** e una **sequenziale**, aggiornate nel rispettivo processo. Per chiarezza, nella tabella 1 riportiamo solo la versione combinatoria poiché la funzione svolta è la medesima. Alcune operazioni sono spezzate su più stati per ragioni hardware, dunque anche i segnali sono espansi con segnali parziali (per dettaglio vedere la sezione 5.2).

Table 1: Tabella delle funzionalità dei segnali

Nome segnale	Funzione
curr_state: S	enum che rappresenta i vari stati, da S_0 a S_{16}
delta_add	contatore per la posizione corrente in memoria, partendo da i_add
j	contatore per la posizione da $-l$ a $+l$
order	ordine del filtro
K	lunghezza della sequenza di parole da elaborare
c_i	7 segnali usati per salvare i coefficienti usati
p_i	7 segnali usati per salvare i prodotti parziali
SUM_partial_i	due segnali per le somme parziali dei p_i
SUM	somma dei SUM_partial_i (=somma finale dei p_i)
sh_i	4 segnali intermedi per gli shift logici
SUM_SHIFT_partial_i	calcolo parziale del valore normalizzato
SUM_SHIFT	valore finale normalizzato
o_done_int	segnale interno mappato sull'uscita o_done
o_mem_data_int	segnale interno mappato sull'uscita o_mem_data
o_mem_en_int	segnale interno mappato sull'uscita o_mem_en
o_mem_we_int	segnale interno mappato sull'uscita o_mem_we
o_mem_addr_int	segnale interno mappato sull'uscita o_mem_addr

Gli ultimi cinque segnali sono utilizzati per l'interazione con la memoria, assegnandoli in modo combinatorio fuori dai processi ai segnali di output.

4 Risultati Funzionali

La nostra architettura funziona sia in **pre-sintesi** che in **post-sintesi**, assicurando un corretto funzionamento anche in tutti i **corner-case** trovati, ovvero:

- **Saturazioni** al limite dell'intervallo $[-128, +127]$ del risultato calcolato, nei quattro casi di Overflow e saturazioni positive e negative, per l'ordine 3 e l'ordine 5.
- **Numero massimo** della lunghezza della sequenza di parole da elaborare ($\frac{(2^{16}-1)-17}{2} = 32759$), per entrambi gli ordini.
- **Elaborazioni successive** di scenari differenti in ogni valore della sequenza di inizializzazione e della sequenza di dati da elaborare (1), con **normale terminazione**.
- **Elaborazioni successive** di scenari differenti in ogni valore della sequenza di inizializzazione e della sequenza di dati da elaborare (1), con **impulsi di reset** nel mezzo dell'elaborazione.
- **Primo e settimo coefficiente diversi da zero** nel caso di elaborazioni del filtro di ordine 3, testato praticamente in tutti i testbench.

Sono stati testati altri testbench del tipo `OrdS_k`, dove S è l'ordine e K è la lunghezza della sequenza di input, che racchiudono i test per il normale funzionamento dell'architettura. Infine, è stato sottoposto anche il testbench d'esempio fornito dal docente al momento della presentazione della specifica. Per analizzare il caso di **normale funzionamento** (con riferimento allo pseudocodice 3) notiamo, come accennato in precedenza, che l'algoritmo ha **complessità** $\mathcal{O}(n)$, dove n è la lunghezza del vettore di input ($n = K + 17$). Tale risultato viene confermato anche nelle nostre simulazioni, come mostrato nel grafico 4 del tempo di simulazione in base alla lunghezza del vettore delle K parole.

Il test è stato effettuato con i seguenti valori di lunghezza di input e ha prodotto i risultati in ms riportati a fianco (non viene fatta distinzione tra ordine 3 e ordine 5 poiché la differenza risulta minima): (24, 0.01551); (100, 0.061170); (500, 0.301170); (1000, 0.601170); (5000, 3.001170); (10000, 6.001170); (15000, 9.001170); (20000, 12.001170); (25000, 15.001170); (32759,

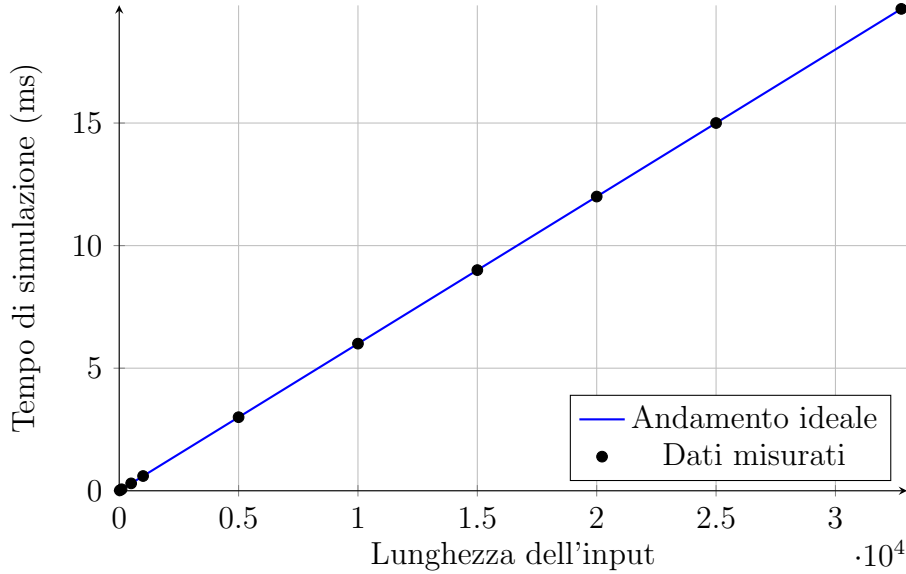


Figure 4: Comparazione andamento temporale teorico e pratico della nostra architettura.

19.656570). Ciò dimostra che la **complessità lineare** teorica è stata mantenuta anche dalla nostra implementazione.

5 Risultati Hardware

In questa sezione tratteremo i **risultati post-sintesi** della nostra architettura, con particolare riferimento all'utilizzo di componenti nella FPGA *Artix-7 xc7a200tbg484-1* e al percorso critico nella rete sintetizzata.

5.1 Report Utilization

Per quanto riguarda l'utilizzo di **componenti** post-sintesi, il nostro obiettivo è stato quello di azzerare il numero di latch utilizzati in modo da evitare transizioni indesiderate sul livello del clock e sincronizzarle sul fronte di salita tramite **flip-flop**.

Nell'elaborazione è stata presa ispirazione dalle linee guida di VIVADO sulla sintesi e le '*HDL Coding Techniques*' [2].

Come precedentemente accennato, abbiamo diviso ogni segnale in due versioni, una sequenziale e una combinatoria, in modo che:

- Entrambi i processi presentano un caso di default in cui ogni segnale combinatorio viene assegnato alla sua versione sequenziale (*delta_function*)

o viceversa (*lambda_function*), in modo che il comportamento sia sempre specificato.

- La *delta_function* modifichi solo il segnale *curr_state*, ad eccezione dell'arrivo di un segnale di reset.
- La *lambda_function* modifichi solo segnali combinatori e legga solo segnali sequenziali, quindi stabili su tutto il livello del clk.

Tramite questa gestione abbiamo ottenuto i seguenti risultati:

- Inferenza di flip-flop per il salvataggio dei segnali.
- Eliminazione di latch indesiderati trovati durante lo sviluppo grazie ad un comportamento determinato in ogni ramo della logica.
- Chiarezza e leggibilità del codice.

L'output della sezione di interesse del comando '*report_utilization*' è il seguente:

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	1117	0	134600	0.83
LUT as Logic	1117	0	134600	0.83
LUT as Memory	0	0	46200	0.00
Slice Registers	346	0	269200	0.13
Register as Flip Flop	346	0	269200	0.13
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

Come si evince dalla figura, il numero di registri latch utilizzati è zero e vengono inferiti solamente flip-flop. Per l'output integrale del comando '*report_utilization*' vedere l'appendice A.1.

5.2 Report Timing

Abbiamo selezionato nello sviluppo della nostra architettura delle operazioni da **dividere e distribuire** su più stati, in modo da ridurre la lunghezza del percorso critico e quindi il suo slack time.

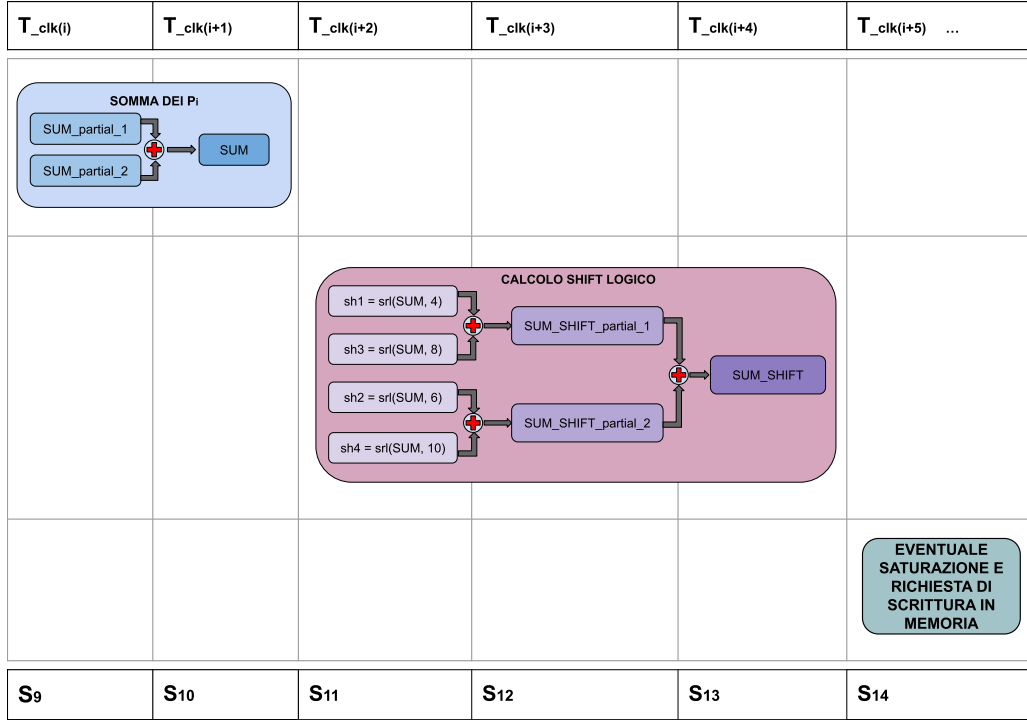


Figure 5: Divisione delle operazioni sui cicli di clock e sugli stati

Tali operazioni sono quelle che richiedono più tempo e le più complesse in termini di fattori, in particolare la somma dei prodotti parziali p_i e gli shift logici. Lo schema in figura 5 mostra la divisione delle operazioni in termini di cicli di clock e di stati.

Questa divisione ci ha consentito di ridurre, nel circuito sintetizzato su FPGA, il *worst negative slack* da 25.374 ns (che costituisce una violazione in quanto il periodo di clock deve durare da specifica al massimo 20 ns) a 12.242 ns.

Slack (MET) : 12.242ns (required time - arrival time)

Questo garantisce il corretto funzionamento del sistema, anche con un periodo di clock fino a 12.243 ns. Il **path critico** risultante coinvolge i segnali `delta_add`, `K_seq` e `p6_seq`, operazioni non ulteriormente frammentabili dello stato S_8 .

Per l'output integrale del comando '*report_timing*' vedere l'appendice A.2.

6 Conclusioni

Il progetto soddisfa tutti i requisiti funzionali indicati nella specifica, avendo coperto tutti i corner case che sono stati trovati. Tutti i test di verifica hanno confermato la correttezza del comportamento del sistema in ogni possibile stato, sia in pre-sintesi che in post-sintesi.

L'utilizzo a livello di componenti hardware è stato, sul modello Artix-7 xc7a200tfbg484-1, di 0.83% di LUT e di 0.13% di *Register as Flip Flop*, con un worst slack time di 12.242 ns, mantenendo la complessità lineare teorica.

Grazie a questo progetto abbiamo avuto l'occasione di mettere in pratica le nozioni imparate durante il corso riguardanti VHDL, FSM e interazioni con la memoria. Abbiamo inoltre imparato l'utilizzo base di uno strumento per la sintesi e la simulazione di circuiti come VIVADO, e acquisito un nuovo modo di pensare alla programmazione.

References

- [1] Xilinx inc. *Vivado Design Suite User Guide UG910 Getting Started*. 2018. URL: <https://docs.amd.com/v/u/2018.3-English/ug910-vivado-getting-started>.
- [2] Xilinx inc. *Vivado Design Suite User Guide: Synthesis (UG901)*. 2025. URL: <https://docs.amd.com/r/en-US/ug901-vivado-synthesis/HDL-Coding-Techniques>.

A Appendice

A.1 Output report_utilization

Utilization Design Information

Table of Contents

- 1. Slice Logic
 - 1.1 Summary of Registers by Type
- 2. Memory
- 3. DSP
- 4. IO and GT Specific
- 5. Clocking
- 6. Specific Feature
- 7. Primitives
- 8. Black Boxes
- 9. Instantiated Netlists

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	1117	0	134600	0.83
LUT as Logic	1117	0	134600	0.83
LUT as Memory	0	0	46200	0.00
Slice Registers	346	0	269200	0.13
Register as Flip Flop	346	0	269200	0.13
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

* Warning! The Final LUT count, after physical optimizations and full implementation, is typically lower. Run opt_design after synthesis, if not already completed, for a more realistic count.

1.1 Summary of Registers by Type

Total	Clock Enable	Synchronous	Asynchronous
0	-	-	-
0	-	-	Set
0	-	-	Reset
0	-	Set	-
0	-	Reset	-
0	Yes	-	-
1	Yes	-	Set
345	Yes	-	Reset
0	Yes	Set	-
0	Yes	Reset	-

2. Memory

Site Type	Used	Fixed	Available	Util%
Block RAM Tile	0	0	365	0.00
RAMB36/FIFO*	0	0	365	0.00
RAMB18	0	0	730	0.00

* Note: Each Block RAM Tile only has one FIFO logic available and therefore can accommodate only one FIFO36E1 or one FIFO18E1. However, if a FIFO18E1 occupies a Block RAM Tile, that tile can still accommodate a RAMB18E1

3. DSP

Site Type	Used	Fixed	Available	Util%
DSPs	0	0	740	0.00

4. IO and GT Specific

Site Type	Used	Fixed	Available	Util%
Bonded IOB	54	0	285	18.95
Bonded IPADs	0	0	14	0.00
Bonded OPADs	0	0	8	0.00
PHY_CONTROL	0	0	10	0.00
PHASER_REF	0	0	10	0.00
OUT_FIFO	0	0	40	0.00
IN_FIFO	0	0	40	0.00
IDELAYCTRL	0	0	10	0.00
IBUFDS	0	0	274	0.00
GTPE2_CHANNEL	0	0	4	0.00
PHASER_OUT/PHASER_OUT_PHY	0	0	40	0.00
PHASER_IN/PHASER_IN_PHY	0	0	40	0.00
IDELAYE2/IDELAYE2_FINEDELAY	0	0	500	0.00
IBUFDS_GTE2	0	0	2	0.00
ILOGIC	0	0	285	0.00
OLOGIC	0	0	285	0.00

5. Clocking

Site Type	Used	Fixed	Available	Util%
BUFGCTRL	1	0	32	3.13
BUFIO	0	0	40	0.00
MMCME2_ADV	0	0	10	0.00
PLLE2_ADV	0	0	10	0.00
BUFMRCE	0	0	20	0.00
BUFHCE	0	0	120	0.00
BUFR	0	0	40	0.00

6. Specific Feature

Site Type	Used	Fixed	Available	Util%
BSCANE2	0	0	4	0.00
CAPTUREE2	0	0	1	0.00
DNA_PORT	0	0	1	0.00
EFUSE_USR	0	0	1	0.00
FRAME_ECCE2	0	0	1	0.00
ICAPE2	0	0	2	0.00
PCIE_2_1	0	0	1	0.00
STARTUPE2	0	0	1	0.00
XADC	0	0	1	0.00

7. Primitives

Ref Name	Used	Functional Category
LUT6	494	LUT
FDCE	345	Flop & Latch
LUT2	309	LUT
LUT4	282	LUT
CARRY4	167	CarryLogic
LUT3	158	LUT
LUT5	147	LUT
OBUF	27	IO
IBUF	27	IO
LUT1	14	LUT
FDPE	1	Flop & Latch
BUFG	1	Clock

8. Black Boxes

Ref Name	Used
----------	------

9. Instantiated Netlists

```

+-----+-----+
| Ref Name | Used |
+-----+-----+

```

A.2 Output report_timing

Timing Report

```

Slack (MET) :           12.242ns  (required time - arrival time)
Source:           delta_add_seq_reg[15]/C
                  (rising edge-triggered cell FDCE clocked by clock
                  {rise@0.000ns fall@10.000ns period=20.000ns})
Destination:      p6_seq_reg[0]/D
                  (rising edge-triggered cell FDCE clocked by clock
                  {rise@0.000ns fall@10.000ns period=20.000ns})
Path Group:       clock
Path Type:        Setup (Max at Slow Process Corner)
Requirement:      20.000ns  (clock rise@20.000ns - clock rise@0.000ns)
Data Path Delay:  7.607ns  (logic 3.082ns (40.515%)  route 4.525ns
(59.485%))
Logic Levels:     10  (CARRY4=4 LUT3=1 LUT4=3 LUT6=2)
Clock Path Skew:  -0.145ns  (DCD - SCD + CPR)
Destination Clock Delay (DCD):    2.100ns = ( 22.100 - 20.000 )
Source Clock Delay   (SCD):    2.424ns
Clock Pessimism Removal (CPR):    0.178ns
Clock Uncertainty:   0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
Total System Jitter   (TSJ):    0.071ns
Total Input Jitter    (TIJ):    0.000ns
Discrete Jitter       (DJ):    0.000ns
Phase Error           (PE):    0.000ns

```

Location	Delay type	Incr(ns)	Path(ns)	Netlist Resource(s)
	(clock clock rise edge)	0.000	0.000 r	
		0.000	0.000 r	i_clk (IN)
net (fo=0)		0.000	0.000	i_clk

IBUF (Prop_ibuf_I_0)	0.944	0.944	r	i_clk_IBUF_inst/0
net (fo=1, unplaced)	0.800	1.744		i_clk_IBUF
BUFG (Prop_bufg_I_0)	0.096	1.840	r	i_clk_IBUF_BUFG_inst/0
net (fo=346, unplaced)	0.584	2.424		i_clk_IBUF_BUFG
FDCE			r	delta_add_seq_reg[15]/C

FDCE (Prop_fdce_C_Q)	0.456	2.880	r	delta_add_seq_reg[15]/Q
net (fo=6, unplaced)	0.773	3.653		delta_add_seq[15]
LUT4 (Prop_lut4_I1_0)	0.295	3.948	f	delta_add_seq[0]_i_5/0
net (fo=6, unplaced)	0.481	4.429		delta_add_seq[0]_i_5_n_0
LUT6 (Prop_lut6_I5_0)	0.124	4.553	r	delta_add_seq[0]_i_3/0
net (fo=33, unplaced)	0.521	5.074		delta_add_seq[0]_i_3_n_0
LUT6 (Prop_lut6_I4_0)	0.124	5.198	f	k_seq[7]_i_3/0
net (fo=9, unplaced)	0.490	5.688		k_seq[7]_i_3_n_0
LUT4 (Prop_lut4_I2_0)	0.124	5.812	r	k_seq[2]_i_1/0
net (fo=3, unplaced)	0.657	6.469		k_seq[2]_i_1_n_0
CARRY4 (Prop_carry4_DI[1]_C0[3])				
	0.507	6.976	r	p6_seq_reg[17]_i_38/C0[3]
net (fo=1, unplaced)	0.009	6.985		p6_seq_reg[17]_i_38_n_0
CARRY4 (Prop_carry4_CI_C0[3])				
	0.114	7.099	r	p6_seq_reg[17]_i_29/C0[3]
net (fo=1, unplaced)	0.000	7.099		p6_seq_reg[17]_i_29_n_0
CARRY4 (Prop_carry4_CI_0[1])				
	0.348	7.447	f	p6_seq_reg[17]_i_28/0[1]
net (fo=2, unplaced)	0.622	8.069		p6_seq_reg[17]_i_28_n_6
LUT4 (Prop_lut4_I1_0)	0.332	8.401	r	p6_seq[17]_i_10/0
net (fo=1, unplaced)	0.000	8.401		p6_seq[17]_i_10_n_0
CARRY4 (Prop_carry4_DI[1]_C0[3])				
	0.534	8.935	f	p6_seq_reg[17]_i_5/C0[3]
net (fo=16, unplaced)	0.972	9.907		p61
LUT3 (Prop_lut3_I2_0)	0.124	10.031	r	p6_seq[0]_i_1/0
net (fo=1, unplaced)	0.000	10.031		p6[0]
FDCE			r	p6_seq_reg[0]/D

(clock clock rise edge)	20.000	20.000	r	
	0.000	20.000	r	i_clk (IN)
net (fo=0)	0.000	20.000		i_clk
IBUF (Prop_ibuf_I_0)	0.811	20.811	r	i_clk_IBUF_inst/0
net (fo=1, unplaced)	0.760	21.570		i_clk_IBUF
BUFG (Prop_bufg_I_0)	0.091	21.661	r	i_clk_IBUF_BUFG_inst/0

net (fo=346, unplaced)	0.439	22.100	i_clk_IBUF_BUFG
FDCE			r p6_seq_reg[0]/C
clock pessimism	0.178	22.279	
clock uncertainty	-0.035	22.243	
FDCE (Setup_fdce_C_D)	0.029	22.272	p6_seq_reg[0]

required time		22.272	
arrival time		-10.031	

slack		12.242	